



**INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO**



Ingeniería en Sistemas Computacionales

SISTEMAS DISTRIBUIDOS

Nombre:

Ávila Cigarroa Aarón Axel

Profesor:

Carreto Arellano Chadwick

Práctica 6: “Servicios Web”

Contenido

Antecedentes.....	3
Problemática	4
Solución	4
Implementación	5
Resultados.....	7
Ilustración 1. Página Principal.....	7
Ilustración 2. Pantalla de consulta de disponibilidad de lugares	8
Ilustración 3. Muestra gráfica del mapa de consulta	8
Ilustración 4. Página para realizar una reservación	9
Ilustración 5. Notificación de reserva exitosa	9
Ilustración 6. Mapa actualizado con reserva.....	10
Ilustración 7. Ventana de reservación fallida	10
Ilustración 8. Página para cancelar reserva	11
Ilustración 9. Ventana de cancelación exitosa	11
Ilustración 10. Mapa actualizado con cancelación de reserva.....	12
Conclusiones.....	12
Anexos	13

Antecedentes

En la etapa anterior del proyecto (Práctica 5), el sistema de reservaciones evolucionó hacia el uso de **Objetos Distribuidos (Java RMI)**. Esta tecnología permitió alcanzar la "Transparencia de Invocación", ocultando la complejidad de los Sockets y permitiendo que el cliente ejecutara métodos remotos como si fueran locales. Sin embargo, RMI impone una restricción arquitectónica severa: un alto nivel de acoplamiento tecnológico. Tanto el cliente como el servidor estaban obligados a estar desarrollados en el lenguaje Java y a compartir las mismas interfaces precompiladas. En el ecosistema tecnológico moderno, los sistemas requieren comunicarse entre diferentes plataformas, lenguajes y dispositivos móviles. Para lograr la **interoperabilidad total**, la arquitectura debe migrar hacia los **Servicios Web**.

Evolución Histórica de los Servicios Web: La necesidad de comunicar sistemas heterogéneos dio origen a los Servicios Web, los cuales han atravesado una notable evolución conceptual:

1. **Los Inicios (XML-RPC y SOAP):** A finales de los años 90 y principios de los 2000, surgieron protocolos como XML-RPC y posteriormente **SOAP (Simple Object Access Protocol)**. Estos protocolos estandarizaron el intercambio de mensajes utilizando el lenguaje de marcado XML. Aunque SOAP dotó a la industria de un estándar robusto y fuertemente tipado (apoyado por contratos WSDL), resultó ser una tecnología pesada, verbosa, difícil de implementar y con un alto consumo de ancho de banda.
2. **El Cambio de Paradigma (REST):** En el año 2000, Roy Fielding presentó en su tesis doctoral la arquitectura **REST (Representational State Transfer)**. REST propuso abandonar la complejidad de SOAP y aprovechar los métodos nativos del protocolo HTTP (GET, POST, PUT, DELETE) para operar sobre "recursos".
3. **La Era Actual (JSON y Microservicios):** Con el auge de las aplicaciones web interactivas y el lenguaje JavaScript, XML fue desplazado por **JSON (JavaScript Object Notation)** como el formato predilecto de intercambio de datos, al ser mucho más ligero y legible. En la actualidad, las API RESTful basadas en JSON son la columna vertebral de la arquitectura de Microservicios, permitiendo que backends robustos (en Node.js, Python, Go, etc.) sirvan datos de manera eficiente a interfaces gráficas web (Single Page Applications) o aplicaciones móviles de manera estandarizada y sin importar el lenguaje de origen.

Problemática

La transición de un sistema basado en Objetos Distribuidos hacia una arquitectura orientada a Servicios Web plantea el reto de resolver la problemática original del proyecto bajo un paradigma tecnológico completamente nuevo:

1. **La Problemática Original (Condiciones de Carrera):** Se debe seguir garantizando la integridad de los datos. En un entorno donde múltiples estudiantes intentan reservar el mismo asiento en la misma sala y horario, el sistema debe prevenir estrictamente la sobreventa (*double-booking*) o el conflicto de recursos.
2. **Desacoplamiento del Lenguaje y Protocolo:** El sistema RMI dependía de la serialización nativa de Java. El nuevo problema consiste en diseñar un medio de comunicación neutral que permita que un cliente (ahora alojado en un navegador web utilizando HTML y JavaScript) se comunique fluidamente con un servidor lógico (Backend) utilizando un formato estandarizado, sin que ninguno conozca la tecnología interna del otro.
3. **Manejo de Estados y Asincronía:** A diferencia de RMI, la arquitectura REST es inherentemente *Stateless* (Sin Estado). El servidor no guarda memoria de conexiones activas; por lo tanto, cada petición HTTP del cliente debe contener toda la información necesaria (ID de sala, asiento, horario) para ser procesada. Asimismo, la nueva interfaz visual exige que las peticiones se realicen de forma asíncrona sin bloquear la pantalla del usuario.

Solución

Para superar estas problemáticas, se propone rediseñar el sistema adoptando una arquitectura moderna dividida estrictamente en un **Backend (Servidor RESTful)** y un **Frontend (Cliente Web Visual)**:

- **Adopción de Node.js y Express:** El servidor central se reescribirá utilizando JavaScript en el entorno de ejecución Node.js con el framework Express. La naturaleza de hilo único (*Single-Threaded Event Loop*) de Node.js actúa como un sincronizador natural; al procesar las peticiones de reserva una por una de manera atómica, se emula la protección de exclusión mutua que originalmente se lograba con la instrucción `synchronized` de Java, erradicando las condiciones de carrera al acceder a la matriz de datos de los asientos.
- **Diseño de un API RESTful con JSON:** La comunicación dejará de depender de interfaces remotas. En su lugar, el servidor expondrá "Puntos de Acceso" (Endpoints) a través de rutas URL HTTP. Toda la información de reservaciones o visualización de mapas se serializará e intercambiará utilizando el formato ligero JSON.
- **Cliente Interactivo (SPA):** La consola de comandos será reemplazada por una interfaz web gráfica interactiva. Se utilizará HTML, CSS y la API `fetch` de JavaScript para consumir los servicios web de manera asíncrona, renderizando visualmente los mapas de disponibilidad (asientos libres y ocupados) y permitiendo al usuario gestionar sus espacios mediante formularios en ventanas modales.

Implementación

El sistema se separó estructuralmente en dos directorios totalmente independientes (backend y frontend), demostrando la interoperabilidad de la arquitectura.

A. El Backend (Servidor de Servicios Web - Express): El motor lógico del servidor se encapsuló en RoomManager.js, el cual mantiene la estructura de datos (5 salas, 36 asientos, horarios). Para exponer esta lógica, se construyó una API mediante Express que integra los siguientes **Servicios Web (Endpoints)**:

- GET /api/salas: Servicio de consulta general. Devuelve un arreglo en formato JSON con el catálogo de las áreas disponibles en el recinto universitario (Biblioteca, Laboratorio, etc.), permitiendo que el cliente construya sus menús dinámicamente.

```
this.salas = [  
  { id: 0, nombre: "Biblioteca", proposito: "Estudio  
Silencioso" },  
  { id: 1, nombre: "Laboratorio", proposito: "Prácticas y  
Actividades" },  
  { id: 2, nombre: "Sala de Descanso", proposito: "Relajación y  
Siestas" },  
  { id: 3, nombre: "Estudio Compartido", proposito: "Reuniones  
Académicas" },  
  { id: 4, nombre: "Sala de Ocio", proposito: "Descanso y  
Diversión" }  
];
```

- GET /api/disponibilidad/:salaId/:dia/:hora: Servicio de consulta de estado. Recibe los parámetros en la URL y retorna un JSON mapeando la matriz gráfica de los 36 asientos, indicando mediante variables booleanas si se encuentran libres u ocupados.

```
crearHorarioVacio() {  
  let horario = [];  
  for (let dia = 0; dia < 5; dia++) {  
    horario[dia] = new Array(12).fill(false); // false = libre  
  }  
  return horario;  
}  
  
consultarMapa(salaId, dia, hora) {  
  let mapa = [];  
  for (let a = 1; a <= 36; a++) {  
    mapa.push({  
      id: a,  
      ocupado: this.asientos[salaId][a][dia][hora]  
    });  
  }  
}
```

```

    }
    return mapa;
}

```

- **POST /api/gestion:** Servicio transaccional. Es el único que utiliza el método HTTP POST. Recibe un *Payload* (cuerpo) en formato JSON con los datos de la operación (accion, salaId, asiento, dia, horaInicio, duracion). Este servicio procesa lógicamente la disponibilidad y consolida la reserva o cancelación en la memoria del servidor, retornando un mensaje de éxito o denegación (ej. estado HTTP 400 por conflictos de ocupación).

```

procesarReserva(salaId, asiento, dia, horaInicio, duracion, esReserva) {
    // Validación de límites
    if (salaId < 0 || salaId > 4 || asiento < 1 || asiento > 36 ||
    horaInicio + duracion > 12) {
        return false;
    }

    let horarioAsiento = this.asientos[salaId][asiento];

    // Verificar disponibilidad previa
    for (let i = 0; i < duracion; i++) {
        let estadoActual = horarioAsiento[dia][horaInicio + i];
        if (esReserva && estadoActual === true) return false; // Ya
ocupado
        if (!esReserva && estadoActual === false) return false; // No
estaba reservado
    }

    // Aplicar cambios
    for (let i = 0; i < duracion; i++) {
        horarioAsiento[dia][horaInicio + i] = esReserva;
    }
    return true;
}

```

B. El Frontend (Cliente de Consumo):

- **Interfaz de Usuario (index.html y style.css):** Se diseñó una interfaz responsiva respetando la paleta institucional (azul y blanco). Se sustituyeron los comandos de texto por selectores dinámicos y un renderizado de cuadrícula (*Grid*) que permite al usuario ver intuitivamente la ocupación de la sala. Se integraron modales estéticos para notificar al usuario sobre el éxito o fallo de la red.

- **Capa de Consumo (app.js):** Actúa como el puente de red. Utiliza la función nativa `fetch` para invocar asincrónicamente las rutas HTTP del servidor. Desempaqueta las respuestas JSON y manipula el DOM (Document Object Model) para actualizar la vista sin necesidad de recargar la página, logrando una experiencia de usuario fluida y transparente.

Resultados

Una vez que hemos inicializado el servidor para que la página cargue y sea visible, nos aparecerá la pantalla principal donde muestra el nombre de la plataforma y las opciones disponibles:

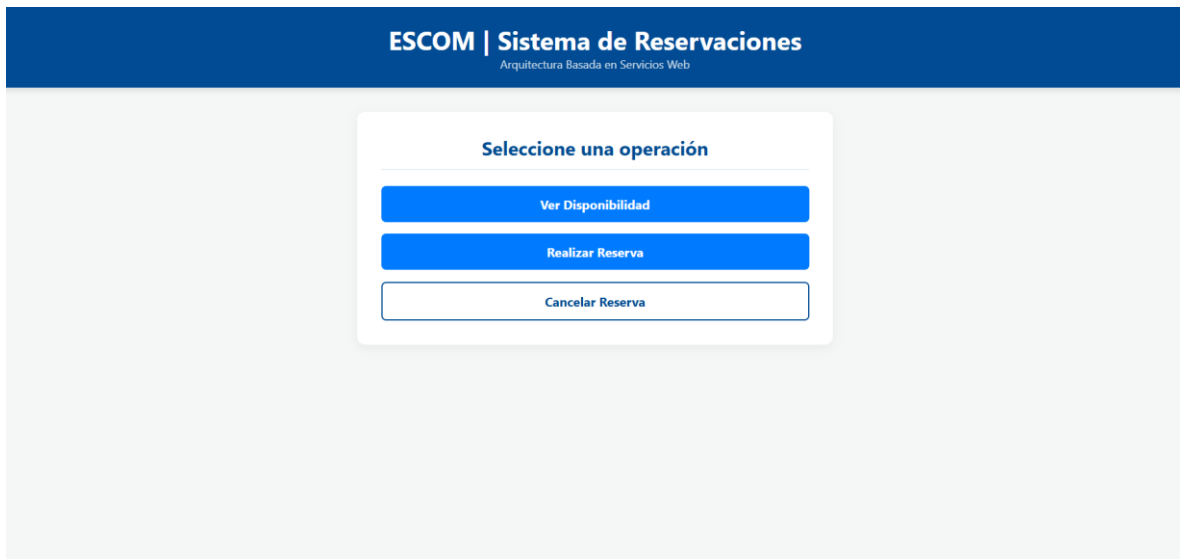


Ilustración 1. Página Principal

Cuando seleccionamos la opción “Ver Disponibilidad”, nos aparecerá una barra de consulta, donde podremos buscar el mapa de la zona en específico que queramos (Biblioteca, Sala de Estudio, etc.) y también podremos escoger el día y horario específico:

The screenshot shows a web interface for the 'ESCOM | Sistema de Reservas' (Architecture Based on Web Services). The main heading is 'Consulta de Disponibilidad'. Below it, there are three dropdown menus: the first is set to 'Biblioteca (Estudio Silencioso)', the second to 'Lunes', and the third to '8:00 hrs'. A blue 'Buscar' button is positioned below these menus. Under the button are two smaller buttons: 'Libre' (white with a blue border) and 'Ocupado' (red with white text). At the bottom of the form is a grey button labeled 'Volver al Menú'.

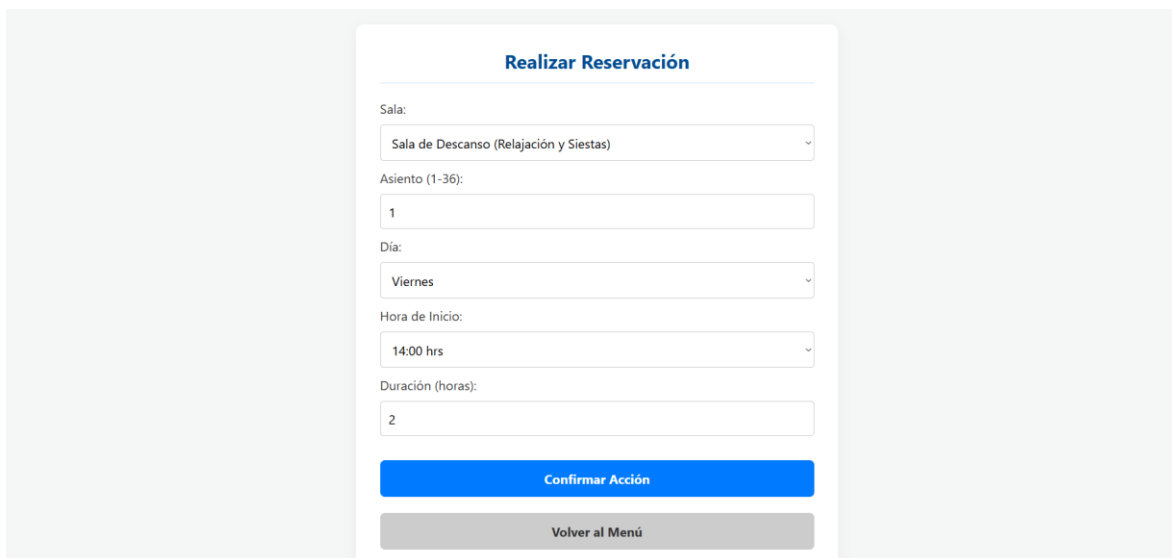
Ilustración 2. Pantalla de consulta de disponibilidad de lugares

Una vez que hemos seleccionado qué mapa queremos visualizar, procedemos a darle al botón “Buscar” para que nos muestre el mapa deseado.

This screenshot shows the same 'Consulta de Disponibilidad' form as in the previous image, but with the 'Buscar' button clicked. Below the 'Libre' and 'Ocupado' buttons, a 6x6 grid of 36 numbered boxes (1 to 36) is displayed. The boxes are arranged in 6 rows and 6 columns, representing a seating chart or map of the selected area.

Ilustración 3. Muestra gráfica del mapa de consulta

Si en la pantalla principal seleccionamos la opción “Realizar Reserva”, nos muestra una nueva pantalla donde podremos seleccionar los datos de la sala que queremos reservar un lugar, tal y como se muestra en la imagen:



Realizar Reservación

Sala:
Sala de Descanso (Relajación y Siestas)

Asiento (1-36):
1

Día:
Viernes

Hora de Inicio:
14:00 hrs

Duración (horas):
2

Confirmar Acción

Volver al Menú

Ilustración 4. Página para realizar una reservación

Si los datos son correctos y no hay conflicto con algún lugar ocupado, nos sale una notificación confirmando la reservación:

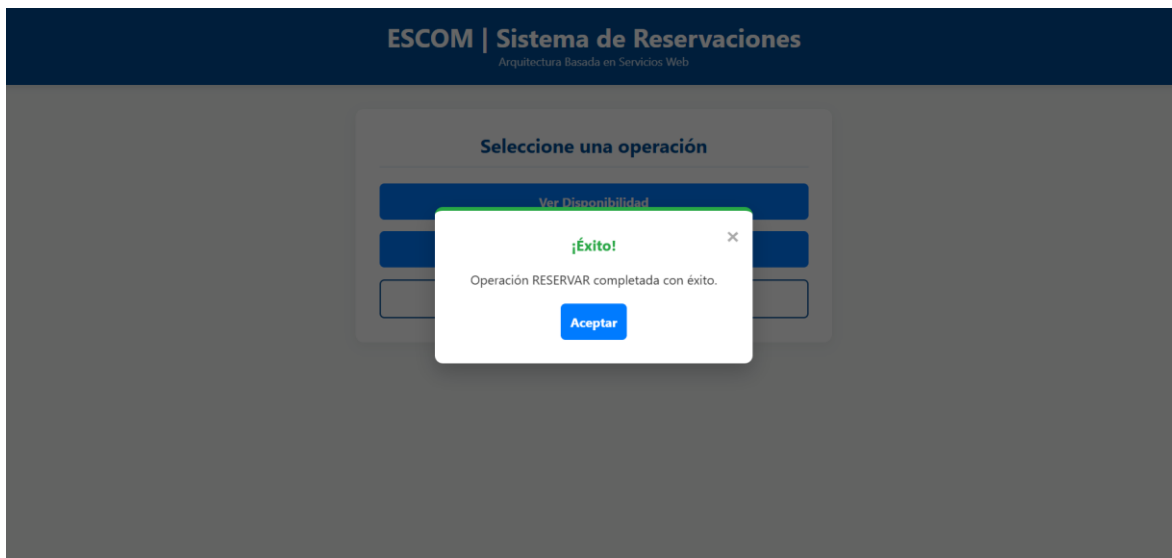


Ilustración 5. Notificación de reserva exitosa

Si procedemos a consultar el mapa actualizado de esa sala en la que hemos reservado el lugar, se mostrará el cambio realizado:

The interface titled "Consulta de Disponibilidad" (Availability Check) shows a dropdown menu for "Sala de Descanso (Relajación y Siestas)", a dropdown for "Viernes", and a dropdown for "14:00 hrs". Below these is a blue "Buscar" button. Underneath the button are two buttons: "Libre" (blue) and "Ocupado" (red). Below these is a 6x6 grid of seats. The seats are numbered 1 to 36. Seats 1, 8, 14, 19, 23, 24, 34, and 36 are marked with a red "X", indicating they are occupied. All other seats are empty, indicating they are free.

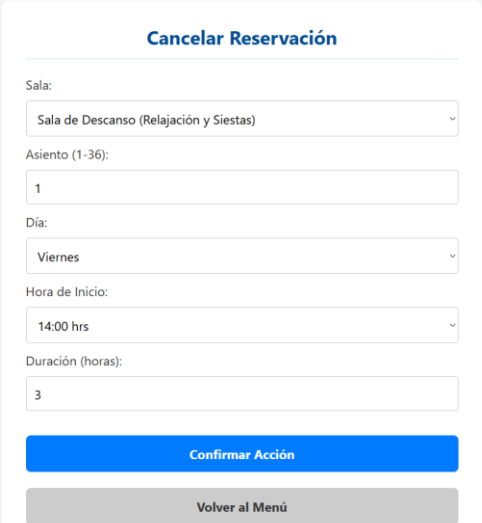
Ilustración 6. Mapa actualizado con reserva

Si queremos reservar un lugar que ya se encuentra ocupado, no procederá la solicitud, y se mostrará un mensaje de error:

The interface titled "Realizar Reservación" (Make Reservation) shows a form with fields for "Sala:", "Asiento (1)", "Día:", "Hora de Inicio:", and "Duración (horas):". The "Sala" field is set to "Sala de...", "Asiento (1)" is set to "24", "Día" is set to "Viernes", "Hora de Inicio" is set to "14:00 hrs", and "Duración (horas)" is set to "3". A blue "Reservar" button is at the bottom. An error message overlay is displayed in the center, with the title "Atención" and the text "Conflicto detectado al reservar." Below the text is a blue "Aceptar" button.

Ilustración 7. Ventana de reservación fallida

También podemos cancelar la reservación de un lugar ocupado, ingresando los datos como se muestra en la imagen:



Formulario para cancelar una reservación. El formulario contiene los siguientes campos:

- Sala:** Seleccionado "Sala de Descanso (Relajación y Siestas)".
- Asiento (1-36):** Seleccionado "1".
- Día:** Seleccionado "Viernes".
- Hora de Inicio:** Seleccionado "14:00 hrs".
- Duración (horas):** Seleccionado "3".

Botones de acción:

- Confirmar Acción** (Botón azul)
- Volver al Menú** (Botón gris)

Ilustración 8. Página para cancelar reserva

Si los datos son correctos, entonces la cancelación será exitosa.

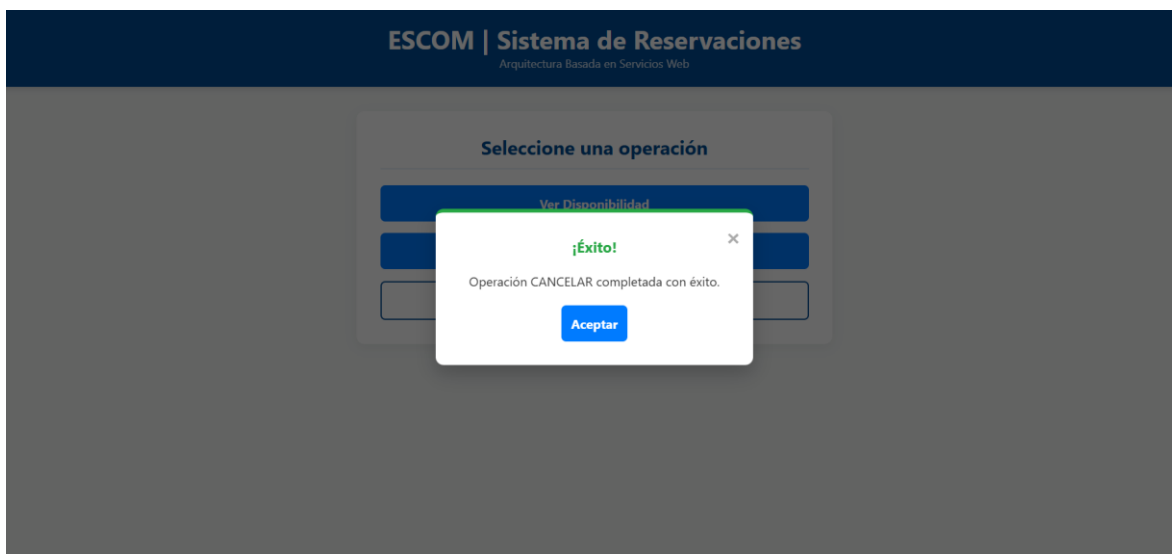


Ilustración 9. Ventana de cancelación exitosa

Si verificamos el mapa donde hemos cancelado la reservación, podremos ver que el cambio se visualiza correctamente.

Consulta de Disponibilidad

Sala de Descanso (Relajación y Siestas)

Viernes

14:00 hrs

Buscar

Libre Ocupado

1	2	3	4	5	6
7	X	9	10	11	12
13	14	15	16	17	18
19	X	21	22	23	X
25	26	27	28	29	30
31	32	33	34	X	36

Ilustración 10. Mapa actualizado con cancelación de reserva

Conclusiones

La evolución de este sistema hacia una arquitectura basada en **Servicios Web** representa el estándar definitivo en la construcción de sistemas informáticos distribuidos modernos. Se comprobó con éxito la viabilidad de separar completamente las tecnologías: un motor lógico operando con Node.js en el backend puede servir de manera impecable y segura a un cliente puramente visual construido con HTML y JavaScript estándar.

El uso de **API RESTful** acoplado al formato **JSON** solucionó de manera contundente las fricciones de las arquitecturas anteriores (como la pesadez del empaquetado manual de Sockets o el acoplamiento estricto de Java RMI). La adopción de los métodos HTTP (GET para obtención segura de mapas y POST para alterar el estado de las reservaciones) dotó al sistema de una semántica universal, haciendo que los servicios web implementados puedan ser consumidos en el futuro no solo por este navegador web, sino por cualquier otro dispositivo o lenguaje de programación.

Finalmente, se demostró que la problemática central de la sincronización de recursos compartidos (la condición de carrera por un asiento) puede ser mitigada exitosamente bajo este paradigma. Al delegar la orquestación transaccional a la naturaleza de un único hilo del *Event Loop* del servidor backend, el sistema es capaz de recibir ráfagas de peticiones de múltiples usuarios a través de la red web sin arriesgar la integridad de la matriz de horarios.

Anexos

Repositorio a GitHub con los archivos de esta práctica:

https://github.com/KrizKs/P6_ServiciosWebACAA.git

Contenido de los archivos en repositorio:

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Reservas ESCOM - Web Services</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>ESCOM | Sistema de Reservaciones</h1>
    <p>Arquitectura Basada en Servicios Web</p>
  </header>

  <main id="app-container">
    <section id="pantalla-menu" class="pantalla activa">
      <h2>Seleccione una operación</h2>
      <div class="botones-menu">
        <button onclick="mostrarPantalla('pantalla-consulta')"
class="btn-primario">Ver Disponibilidad</button>
        <button onclick="prepararFormulario('RESERVAR')" class="btn-
primario">Realizar Reserva</button>
        <button onclick="prepararFormulario('CANCELAR')" class="btn-
secundario">Cancelar Reserva</button>
      </div>
    </section>

    <section id="pantalla-consulta" class="pantalla oculta">
      <h2>Consulta de Disponibilidad</h2>
      <div class="controles">
        <select id="sel-sala-consulta"></select>
        <select id="sel-dia-consulta">
          <option value="0">Lunes</option>
          <option value="1">Martes</option>
          <option value="2">Miércoles</option>
        </select>
      </div>
    </section>
  </main>
</body>
</html>
```

```

        <option value="3">Jueves</option>
        <option value="4">Viernes</option>
    </select>
    <select id="sel-hora-consulta">
    </select>
    <button onclick="consultarMapa()" class="btn-
primario">Buscar</button>
    </div>

    <div id="mapa-container" class="oculta">
        <div class="leyenda">
            <span class="seat libre-ejemplo">Libre</span>
            <span class="seat ocupado-ejemplo">Ocupado</span>
        </div>
        <div id="grid-asientos" class="grid"></div>
    </div>
    <button onclick="mostrarPantalla('pantalla-menu')" class="btn-
volver mt-2">Volver al Menú</button>
</section>

<section id="pantalla-formulario" class="pantalla oculta">
    <h2 id="titulo-formulario">Gestión</h2>
    <form id="form-gestion" onsubmit="enviarGestion(event)">
        <input type="hidden" id="input-accion">

        <label>Sala:</label>
        <select id="sel-sala-form" required></select>

        <label>Asiento (1-36):</label>
        <input type="number" id="input-asiento" min="1" max="36"
required>

        <label>Día:</label>
        <select id="sel-dia-form" required>
            <option value="0">Lunes</option><option
value="1">Martes</option>
            <option value="2">Miércoles</option><option
value="3">Jueves</option>
            <option value="4">Viernes</option>
        </select>

        <label>Hora de Inicio:</label>
        <select id="sel-hora-form" required></select>

        <label>Duración (horas):</label>

```

```

        <input type="number" id="input-duracion" min="1" max="4"
required>

        <button type="submit" class="btn-primario mt-2">Confirmar
Acción</button>
    </form>
    <button onclick="mostrarPantalla('pantalla-menu')" class="btn-
volver mt-2">Volver al Menú</button>
</section>
<div id="modal-alerta" class="modal oculta">
    <div class="modal-contenido">
        <span class="cerrar-modal"
onclick="cerrarAlerta()">&times;</span>
        <h3 id="modal-titulo">Notificación</h3>
        <p id="modal-mensaje" class="mt-2"></p>
        <button onclick="cerrarAlerta()" class="btn-primario mt-
2">Aceptar</button>
    </div>
</div>
</main>

<script src="app.js"></script>
</body>
</html>

```

style.css

```

:root {
    --primary-blue: #004b93;
    --light-blue: #e6f0fa;
    --accent-blue: #007bff;
    --white: #ffffff;
    --text-dark: #333333;
    --danger: #dc3545;
}

* { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI',
Tahoma, Geneva, Verdana, sans-serif; }

body { background-color: #f4f7f6; color: var(--text-dark); display: flex;
flex-direction: column; align-items: center; }

```

```

header { background-color: var(--primary-blue); color: var(--white); width:
100%; padding: 20px; text-align: center; box-shadow: 0 4px 6px
rgba(0,0,0,0.1); }
header p { font-size: 0.9em; opacity: 0.8; }

#app-container { background: var(--white); width: 90%; max-width: 600px;
margin-top: 30px; padding: 30px; border-radius: 10px; box-shadow: 0 4px 15px
rgba(0,0,0,0.05); }

/* Navegación entre pantallas */
.pantalla.oculta { display: none; }
.pantalla.activa { display: block; animation: fadeIn 0.3s; }
@keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }

h2 { color: var(--primary-blue); border-bottom: 2px solid var(--light-blue);
padding-bottom: 10px; margin-bottom: 20px; text-align: center; }

/* Botones */
.botones-menu { display: flex; flex-direction: column; gap: 15px; }
button { padding: 12px; border: none; border-radius: 6px; font-size: 16px;
cursor: pointer; transition: 0.3s; font-weight: bold; }
.btn-primario { background-color: var(--accent-blue); color: var(--white); }
.btn-primario:hover { background-color: var(--primary-blue); }
.btn-secundario { background-color: var(--white); color: var(--primary-
blue); border: 2px solid var(--primary-blue); }
.btn-secundario:hover { background-color: var(--light-blue); }
.btn-volver { background-color: #ccc; color: #333; width: 100%; }
.btn-volver:hover { background-color: #bbb; }

.mt-2 { margin-top: 20px; }

/* Formularios */
.controles, form { display: flex; flex-direction: column; gap: 10px; }
select, input { padding: 10px; border: 1px solid #ccc; border-radius: 4px;
font-size: 16px; }

/* Grid del Mapa Visual */
#mapa-container { margin-top: 20px; text-align: center; }
.grid { display: grid; grid-template-columns: repeat(6, 1fr); gap: 10px;
justify-items: center; margin-top: 15px; padding: 15px; background: var(--
light-blue); border-radius: 8px;}
.seat { width: 40px; height: 40px; display: flex; align-items: center;
justify-content: center; font-weight: bold; border-radius: 5px; border: 2px
solid var(--primary-blue); background: var(--white); color: var(--primary-
blue); transition: 0.2s;}

```



```

.seat.ocupado { background: var(--danger); border-color: darkred; color:
white; }

.leyenda { display: flex; justify-content: center; gap: 20px; font-size:
14px; }
.libre-ejemplo { width: auto; padding: 2px 10px; }
.ocupado-ejemplo { width: auto; padding: 2px 10px; background: var(--
danger); border-color: darkred; color: white;}

.modal {
  display: none; /* Oculto por defecto */
  position: fixed;
  z-index: 1000;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.6); /* Fondo oscuro translúcido */
  align-items: center;
  justify-content: center;
}

/* Usamos display flex cuando está activo para centrar el contenido */
.modal.activa {
  display: flex;
  animation: fadeIn 0.3s;
}

.modal-contenido {
  background-color: var(--white);
  padding: 30px;
  border-radius: 10px;
  width: 90%;
  max-width: 400px;
  text-align: center;
  box-shadow: 0 10px 25px rgba(0,0,0,0.2);
  position: relative;
  border-top: 5px solid var(--primary-blue);
}

.cerrar-modal {
  color: #aaaaaa;
  position: absolute;
  top: 10px;
  right: 15px;

```

```

    font-size: 28px;
    font-weight: bold;
    cursor: pointer;
    transition: 0.2s;
}

.cerrar-modal:hover {
    color: var(--danger);
}

/* Temas dinámicos para el modal */
.modal-exito { border-top-color: #28a745; }
.modal-exito h3 { color: #28a745; border-bottom: none; margin-bottom: 0;}
.modal-error { border-top-color: var(--danger); }
.modal-error h3 { color: var(--danger); border-bottom: none; margin-bottom: 0;}

```

app.js

```

const API_URL = 'http://localhost:5000/api';

// Inicialización
document.addEventListener('DOMContentLoaded', async () => {
    cargarHoras('sel-hora-consulta');
    cargarHoras('sel-hora-form');
    await cargarSalas();
});

// Llenar selects de hora (8 a 19 hrs)
function cargarHoras(elementId) {
    const select = document.getElementById(elementId);
    for (let i = 0; i < 12; i++) {
        let horaReal = i + 8;
        select.innerHTML += `<option value="${i}">${horaReal}:00
hrs</option>`;
    }
}

// Obtener catálogo de salas mediante GET
async function cargarSalas() {
    try {
        const response = await fetch(`${API_URL}/salas`);
        const salas = await response.json();
    }
}

```

```

        let opciones = '';
        salas.forEach(sala => {
            opciones += `<option value="${sala.id}">${sala.nombre}
(${sala.proposito})</option>`;
        });

        document.getElementById('sel-sala-consulta').innerHTML = opciones;
        document.getElementById('sel-sala-form').innerHTML = opciones;
    } catch (error) {
        console.error("Error al cargar salas (¿El backend está encendido?):", error);
    }
}

// Navegación de interfaz
function mostrarPantalla(idPantalla) {
    document.querySelectorAll('.pantalla').forEach(p => {
        p.classList.remove('activa');
        p.classList.add('oculta');
    });
    document.getElementById(idPantalla).classList.remove('oculta');
    document.getElementById(idPantalla).classList.add('activa');

    if(idPantalla === 'pantalla-menu') {
        document.getElementById('mapa-container').classList.add('oculta');
    }
}

function prepararFormulario(accion) {
    document.getElementById('titulo-formulario').innerText = accion ===
'RESERVAR' ? 'Realizar Reservación' : 'Cancelar Reservación';
    document.getElementById('input-accion').value = accion;
    mostrarPantalla('pantalla-formulario');
}

// GET: Consultar y Dibujar Mapa
async function consultarMapa() {
    const salaId = document.getElementById('sel-sala-consulta').value;
    const dia = document.getElementById('sel-dia-consulta').value;
    const hora = document.getElementById('sel-hora-consulta').value;

    try {
        const response = await
fetch(`${API_URL}/disponibilidad/${salaId}/${dia}/${hora}`);

```

```

const mapa = await response.json();

const grid = document.getElementById('grid-asientos');
grid.innerHTML = ''; // Limpiar mapa anterior

mapa.forEach(asiento => {
    const div = document.createElement('div');
    div.className = `seat ${asiento.ocupado ? 'ocupado' : ''}`;
    div.innerText = asiento.ocupado ? 'X' : asiento.id;
    grid.appendChild(div);
});

document.getElementById('mapa-
container').classList.remove('oculta');
} catch (error) {
    mostrarAlerta("Error al consultar el mapa. Verifique la conexión con
el servidor.", "error");
}
}

// POST: Enviar petición de reserva/cancelación
async function enviarGestion(event) {
    event.preventDefault(); // Evitar que la página se recargue

    const payload = {
        accion: document.getElementById('input-accion').value,
        salaId: parseInt(document.getElementById('sel-sala-form').value),
        asiento: parseInt(document.getElementById('input-asiento').value),
        dia: parseInt(document.getElementById('sel-dia-form').value),
        horaInicio: parseInt(document.getElementById('sel-hora-
form').value),
        duracion: parseInt(document.getElementById('input-duracion').value)
    };

    try {
        const response = await fetch(`${API_URL}/gestion`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });

        const result = await response.json();

        if (response.ok) {
            // Reemplazo del alert de éxito

```

```

        mostrarAlerta(result.message, "exito");
        document.getElementById('form-gestion').reset();
        mostrarPantalla('pantalla-menu');
    } else {
        // Reemplazo del alert de error
        mostrarAlerta(result.message, "error");
    }
} catch (error) {
    // Reemplazo del alert de caída del servidor
    mostrarAlerta("No se pudo conectar con el servidor web.", "error");
}
}

function mostrarAlerta(mensaje, tipo) {
    const modal = document.getElementById('modal-alerta');
    const titulo = document.getElementById('modal-titulo');
    const texto = document.getElementById('modal-mensaje');
    const contenido = document.querySelector('.modal-contenido');

    // Limpiamos estilos anteriores
    contenido.classList.remove('modal-exito', 'modal-error');
    texto.innerText = mensaje;

    // Configuramos el estilo según el tipo de alerta
    if (tipo === 'exito') {
        titulo.innerText = '¡Éxito!';
        contenido.classList.add('modal-exito');
    } else {
        titulo.innerText = 'Atención';
        contenido.classList.add('modal-error');
    }

    // Mostramos el modal
    modal.classList.remove('oculta');
    modal.classList.add('activa');
}

function cerrarAlerta() {
    const modal = document.getElementById('modal-alerta');
    modal.classList.remove('activa');
    modal.classList.add('oculta');
}

```

server.js

```
const express = require('express');
const cors = require('cors');
const roomManager = require('./RoomManager');

const app = express();
const PORT = 5000;

// Middlewares
app.use(cors()); // Permite que el frontend (en otro puerto) haga peticiones
app.use(express.json()); // Permite recibir datos en formato JSON

// Ruta 1: Obtener las salas disponibles
app.get('/api/salas', (req, res) => {
  res.json(roomManager.salas);
});

// Ruta 2: Consultar disponibilidad de una sala específica
app.get('/api/disponibilidad/:salaId/:dia/:hora', (req, res) => {
  const { salaId, dia, hora } = req.params;
  const mapa = roomManager.consultarMapa(parseInt(salaId), parseInt(dia),
  parseInt(hora));
  res.json(mapa);
});

// Ruta 3: Realizar o cancelar una reserva
app.post('/api/gestion', (req, res) => {
  const { accion, salaId, asiento, dia, horaInicio, duracion } = req.body;

  const esReserva = accion === 'RESERVAR';
  const exito = roomManager.procesarReserva(salaId, asiento, dia,
  horaInicio, duracion, esReserva);

  if (exito) {
    res.json({ success: true, message: `Operación ${accion} completada
con éxito.` });
  } else {
    res.status(400).json({ success: false, message: `Conflicto detectado
al ${accion.toLowerCase()}.` });
  }
});

app.listen(PORT, () => {
```

```
console.log(`Servidor de Web Services escuchando en  
http://localhost:${PORT}`);  
});
```

RoomManager.js

```
class RoomManager {  
  constructor() {  
    this.salas = [  
      { id: 0, nombre: "Biblioteca", proposito: "Estudio Silencioso"  
},  
      { id: 1, nombre: "Laboratorio", proposito: "Prácticas y  
Actividades" },  
      { id: 2, nombre: "Sala de Descanso", proposito: "Relajación y  
Siestas" },  
      { id: 3, nombre: "Estudio Compartido", proposito: "Reuniones  
Académicas" },  
      { id: 4, nombre: "Sala de Ocio", proposito: "Descanso y  
Diversión" }  
    ];  
  
    // Inicializamos los asientos: 5 salas x 36 asientos x 5 días x 12  
    horas  
    this.asientos = {};  
    for (let s = 0; s < 5; s++) {  
      this.asientos[s] = {};  
      for (let a = 1; a <= 36; a++) {  
        this.asientos[s][a] = this.crearHorarioVacio();  
      }  
    }  
  
    crearHorarioVacio() {  
      let horario = [];  
      for (let dia = 0; dia < 5; dia++) {  
        horario[dia] = new Array(12).fill(false); // false = libre  
      }  
      return horario;  
    }  
  
    consultarMapa(salaId, dia, hora) {  
      let mapa = [];  
      for (let a = 1; a <= 36; a++) {
```

```

        mapa.push({
            id: a,
            ocupado: this.asientos[salaId][a][dia][hora]
        });
    }
    return mapa;
}

// Node.js es single-threaded, por lo que estas operaciones son atómicas
// naturalmente
procesarReserva(salaId, asiento, dia, horaInicio, duracion, esReserva) {
    // Validación de límites
    if (salaId < 0 || salaId > 4 || asiento < 1 || asiento > 36 ||
    horaInicio + duracion > 12) {
        return false;
    }

    let horarioAsiento = this.asientos[salaId][asiento];

    // Verificar disponibilidad previa
    for (let i = 0; i < duracion; i++) {
        let estadoActual = horarioAsiento[dia][horaInicio + i];
        if (esReserva && estadoActual === true) return false; // Ya
        ocupado
        if (!esReserva && estadoActual === false) return false; // No
        estaba reservado
    }

    // Aplicar cambios
    for (let i = 0; i < duracion; i++) {
        horarioAsiento[dia][horaInicio + i] = esReserva;
    }
    return true;
}
}

module.exports = new RoomManager();

```